Human M1 10x Cell Type Classification with 1D CNN

Dataset: Allen Brain Atlas — Human Primary Motor Cortex (M1) 10x Chromium
Task: Predict cell type from single-cell RNA-seq gene expression

Pipeline:

1. Download gene expression matrix + cell metadata
2. Train / Validation / Test split (on raw counts, before any preprocessing)
3. Preprocess: discover gene filter + HVGs on validation set, then apply to train and test
4. Visualizations on training set (PCA, UMAP, distributions)
5. 1D CNN model (PyTorch) — train and evaluate

1. Imports

```
import os
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from tqdm import tqdm
from pathlib import Path

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

import umap

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import scanpy as sc
import polars as pl

from torch.utils.tensorboard import SummaryWriter

SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_available() else 'cpu')
print(f'Using device: {DEVICE}')
print(f'PyTorch version: {torch.__version__}')
```

```
Using device: cuda
PyTorch version: 2.10.0+cu128
```

2. Download Data

- **Matrix:** Gene expression counts (cells × genes)
- **Metadata:** Cell annotations including cell type labels

Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_rem
```

```
COLAB_DATA_DIR = '/content/drive/MyDrive/FinalProject/data'
MATRIX_URL = 'https://idk-etl-prod-download-bucket.s3.amazonaws.com/aibs_human_m1_10x/matrix.csv'
META_URL   = 'https://idk-etl-prod-download-bucket.s3.amazonaws.com/aibs_human_m1_10x/metadata.csv'

os.makedirs(COLAB_DATA_DIR, exist_ok=True)

PATHS = {
    'matrix':   os.path.join(COLAB_DATA_DIR, 'matrix.csv'),
    'metadata': os.path.join(COLAB_DATA_DIR, 'metadata.csv'),
}

print(f'Data directory: {COLAB_DATA_DIR}')
print(f'Files: {list(PATHS.keys())}')
```

```
Data directory: /content/drive/MyDrive/FinalProject/data
Files: ['matrix', 'metadata']
```

3. Load and Inspect Data

```
print('Loading metadata...')
meta = pd.read_csv(PATHS['metadata'])
print(f'Metadata shape: {meta.shape}')
print(f'Columns: {list(meta.columns)}')
meta.head(3)
```

```
Loading metadata...
Metadata shape: (76533, 39)
Columns: ['sample_name', 'exp_component_name', 'cluster_label', 'cluster_color', 'cluster_order', 'class_label', 'cla
```

	sample_name	exp_component_name	cluster_label	cluster_color	cluster_order	class_label	class_color	cl
0	AAACCCAAGGATTTCC-LKTX_190129_01_A01	AAACCCAAGGATTTCC-21L8TX_180927_001_A01	Inh L1-2 SST CCNJL	#fb8d00	50	GABAergic	#FF7373	
1	AAACCCAAGTATGGCG-LKTX_190129_01_A01	AAACCCAAGTATGGCG-21L8TX_180927_001_A01	Exc L5-6 FEZF2 IFNG-AS1	#2c815f	116	Glutamatergic	#3DCC3D	
2	AAACCCACAAAGTGTA-LKTX_190129_01_A01	AAACCCACAAAGTGTA-21L8TX_180927_001_A01	Exc L3-5 RORB LINC01202	#547d7a	87	Glutamatergic	#3DCC3D	

3 rows × 39 columns

```
expr_raw = pl.read_csv(PATHS['matrix'], infer_schema=1000)
```

```
expr_raw = expr_raw.to_pandas()
```

```
if "sample_name" in expr_raw.columns:
   expr_raw.set_index('sample_name', inplace=True)
```

```
expr_raw.fillna(0, inplace=True)
```

```
label_col = "subclass_label"
id_col = "sample_name"
print(f'\nCell type distribution (top 15):')
print(meta[label_col].value_counts())
```

```
Cell type distribution (top 15):
subclass_label
L2/3 IT       24231
L5 IT         13834
Pvalb          7782
Sst            5936
Vip            4858
Lamp5          4454
L6 CT          3734
Oligo          2942
L6b            2236
L6 IT          1829
L5/6 NP        1487
Sncg            895
L5 ET           858
Astro           568
L6 IT Car3      327
```

```
OPC             283
Micro-PVM       108
Sst Chodl        67
Endo             64
VLMC             40
Name: count, dtype: int64
```

```
meta_indexed = meta.set_index(id_col)
common_ids = expr_raw.index.intersection(meta_indexed.index)
print(f'Cells in matrix:   {len(expr_raw):,}')
print(f'Cells in metadata: {len(meta_indexed):,}')
print(f'Cells in common:   {len(common_ids):,}')

expr = expr_raw.loc[common_ids]
labels_series = meta_indexed.loc[common_ids, label_col]
valid_mask = labels_series.notna()
expr = expr[valid_mask]
labels_series = labels_series[valid_mask]

print(f'\nFinal dataset: {expr.shape[0]:,} cells, {expr.shape[1]:,} genes')
print(f'Number of cell types: {labels_series.nunique()}')
```

```
Cells in matrix:   3,060
Cells in metadata: 76,533
Cells in common:   3,060

Final dataset: 3,060 cells, 50,281 genes
Number of cell types: 19
```

4. Train / Validation / Test Split

Split **before** any preprocessing to prevent data leakage.

Stratified by cell type. Rare classes (< `min_cells_per_class` cells) are dropped first.

```
# --- Filter rare cell types ---
min_cells_per_class_val = 100
class_counts = labels_series.value_counts()
valid_classes = class_counts[class_counts >= min_cells_per_class_val].index

filtered_mask = labels_series.isin(valid_classes)
expr_filtered = expr[filtered_mask]
labels_filtered = labels_series[filtered_mask]

# Encode labels
le = LabelEncoder()
y_all = le.fit_transform(labels_filtered.values)
class_names = le.classes_
n_classes = len(class_names)

print(f'Cells after class filtering: {expr_filtered.shape[0]:,} (from {expr.shape[0]:,})')
print(f'Cell types retained: {n_classes}  {list(class_names)}')

TRAIN_FRAC = 0.80
VAL_FRAC   = 0.10
TEST_FRAC  = 0.10

X_train_raw, X_temp_raw, y_train, y_temp = train_test_split(
    expr_filtered.values, y_all,
    test_size=VAL_FRAC + TEST_FRAC,
    stratify=y_all,
    random_state=SEED
)
X_val_raw, X_test_raw, y_val, y_test = train_test_split(
    X_temp_raw, y_temp,
    test_size=TEST_FRAC / (VAL_FRAC + TEST_FRAC),
    stratify=y_temp,
    random_state=SEED
)

all_gene_names = expr_filtered.columns.tolist()

print(f'\nTrain:      {X_train_raw.shape[0]:>6,} cells  ({X_train_raw.shape[0]/len(y_all)*100:.1f}%)')
print(f'Validation: {X_val_raw.shape[0]:>6,} cells  ({X_val_raw.shape[0]/len(y_all)*100:.1f}%)')
print(f'Test:       {X_test_raw.shape[0]:>6,} cells  ({X_test_raw.shape[0]/len(y_all)*100:.1f}%)')
```

```
Cells after class filtering: 2,807 (from 3,060)
Cell types retained: 9  ['L2/3 IT', 'L5 IT', 'L6 CT', 'L6b', 'Lamp5', 'Oligo', 'Pvalb', 'Sst', 'Vip']

Train:       2,245 cells  (80.0%)
Validation:    281 cells  (10.0%)
Test:          281 cells  (10.0%)
```

```
print(f'Classes ({n_classes} total):')
for i, name in enumerate(class_names):
    print(f'  {i:>3}: {name}')

print(f'\nPer-split class distribution:')
for split_name, y_split in [('Train', y_train), ('Val', y_val), ('Test', y_test)]:
    counts = np.bincount(y_split, minlength=n_classes)
    print(f'  {split_name}: {dict(zip(class_names, counts))}')
```

```
Classes (9 total):
    0: L2/3 IT
    1: L5 IT
    2: L6 CT
    3: L6b
    4: Lamp5
    5: Oligo
    6: Pvalb
    7: Sst
    8: Vip

Per-split class distribution:
  Train: {'L2/3 IT': np.int64(655), 'L5 IT': np.int64(553), 'L6 CT': np.int64(153), 'L6b': np.int64(106), 'Lamp5': np
  Val: {'L2/3 IT': np.int64(82), 'L5 IT': np.int64(69), 'L6 CT': np.int64(19), 'L6b': np.int64(13), 'Lamp5': np.int64
  Test: {'L2/3 IT': np.int64(82), 'L5 IT': np.int64(70), 'L6 CT': np.int64(19), 'L6b': np.int64(13), 'Lamp5': np.int6
```

5. Preprocessing

Preprocessing parameters are discovered on the **validation set**, then applied to train and test.

1. **Gene filtering** — retain genes expressed in ≥ `min_cells_per_gene` cells (threshold scaled to val set size)
2. **Library-size normalization** + log1p (applied per-cell, independently per split)
3. **HVG selection** — top `N_HVG` genes by variance, ranked on val
4. **StandardScaler** — fit on training set, applied to all splits

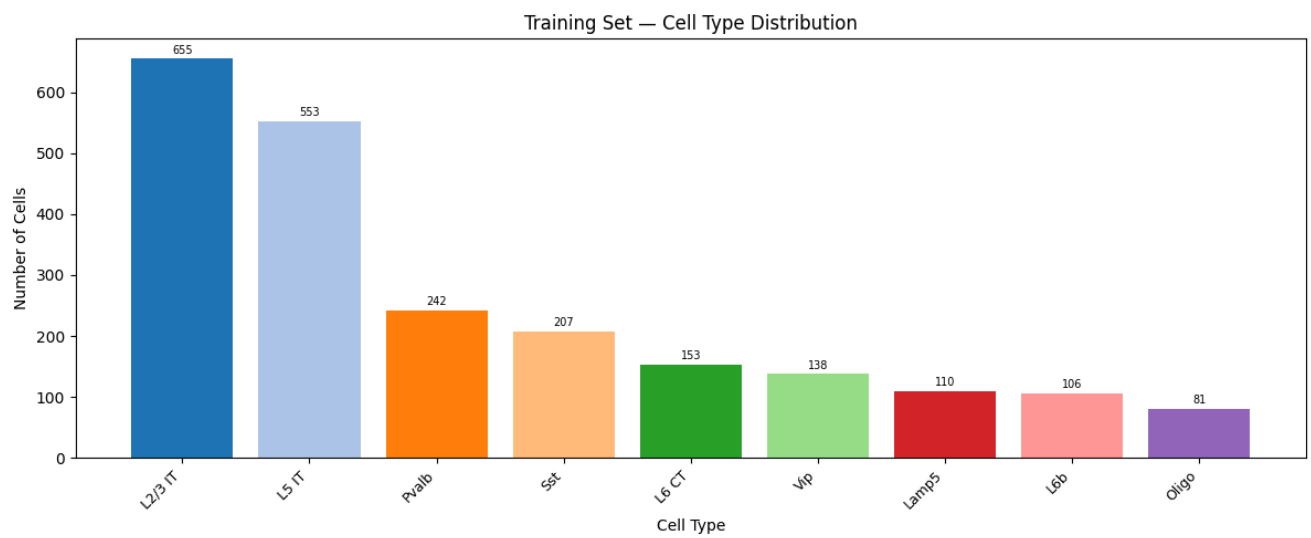
```
N_HVG = 200

min_gene_frac = 0.036
min_cells_per_gene_val = max(3, int(min_gene_frac * X_val_raw.shape[0]))
print(f'Gene filter threshold for val ({X_val_raw.shape[0]} cells): {min_cells_per_gene_val} cells')
val_gene_mask = (X_val_raw > 0).sum(axis=0) >= min_cells_per_gene_val
gene_indices = np.where(val_gene_mask)[0]
print(f'Genes retained after filtering on val: {len(gene_indices):,} (of {X_val_raw.shape[1]:,})')

def normalize_log(X_raw, gene_idx):
    """Library-size normalize and log1p transform selected genes."""
    X_filt = X_raw[:, gene_idx].astype(np.float32)
    lib_sizes = X_filt.sum(axis=1, keepdims=True)
    lib_sizes = np.where(lib_sizes == 0, 1, lib_sizes)
    return np.log1p((X_filt / lib_sizes) * 1e4)

X_val_norm = normalize_log(X_val_raw, gene_indices)
gene_vars = X_val_norm.var(axis=0)
top_hvg_idx = np.argsort(gene_vars)[::-1][:N_HVG]
gene_names_hvg = [all_gene_names[gene_indices[i]] for i in top_hvg_idx]

X_val_hvg = X_val_norm[:, top_hvg_idx]
print(f'Selected {N_HVG} HVGs from val set')

X_train_norm = normalize_log(X_train_raw, gene_indices)
X_train_hvg = X_train_norm[:, top_hvg_idx]

X_test_norm = normalize_log(X_test_raw, gene_indices)
X_test_hvg = X_test_norm[:, top_hvg_idx]
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train_hvg).astype(np.float32)
X_val_s   = scaler.transform(X_val_hvg).astype(np.float32)
X_test_s  = scaler.transform(X_test_hvg).astype(np.float32)
```

```
print(f'\nFinal shapes:')
print(f'  Train: {X_train_s.shape}')
print(f'  Val:   {X_val_s.shape}')
print(f'  Test:  {X_test_s.shape}')
```

```
Gene filter threshold for val (281 cells): 10 cells
Genes retained after filtering on val: 16,448 (of 50,281)
Selected 200 HVGs from val set

Final shapes:
  Train: (2245, 200)
  Val:   (281, 200)
  Test:  (281, 200)
```

6. Training Set Visualizations

```
train_labels = le.inverse_transform(y_train)
label_counts = pd.Series(train_labels).value_counts()
fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.bar(range(len(label_counts)), label_counts.values,
              color=sns.color_palette('tab20', len(label_counts)))
ax.set_xticks(range(len(label_counts)))
ax.set_xticklabels(label_counts.index, rotation=45, ha='right', fontsize=9)
ax.set_xlabel('Cell Type')
ax.set_ylabel('Number of Cells')
ax.set_title('Training Set — Cell Type Distribution')
for bar, count in zip(bars, label_counts.values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
            str(count), ha='center', va='bottom', fontsize=7)
plt.tight_layout()
plt.savefig('fig_class_distribution.png', dpi=150)
plt.show()
print('Saved fig_class_distribution.png')
```

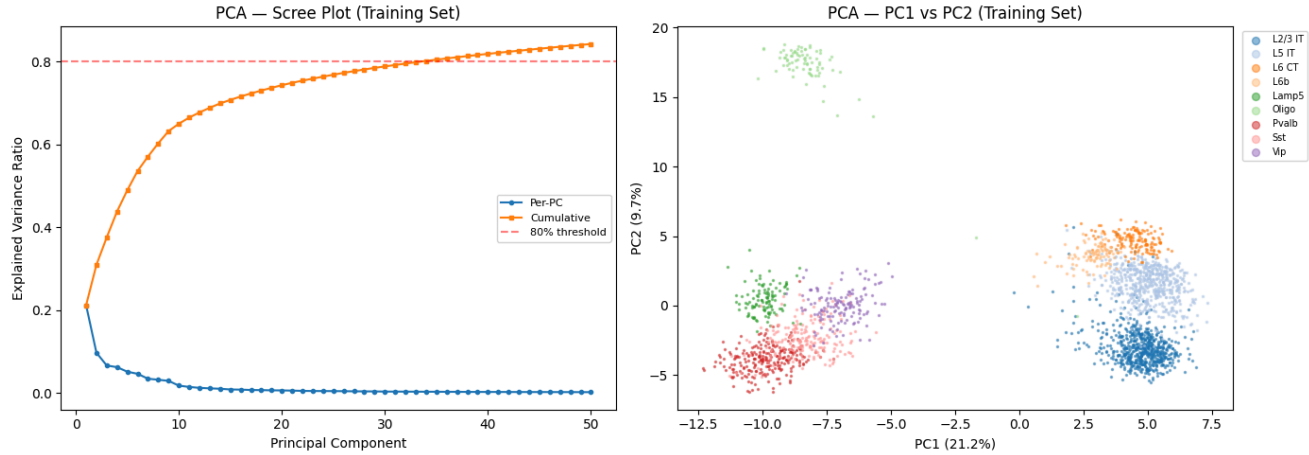



```
Saved fig_class_distribution.png
```

```
print('Running PCA on training set...')
pca = PCA(n_components=50, random_state=SEED)
X_train_pca = pca.fit_transform(X_train_s)

explained_var = np.cumsum(pca.explained_variance_ratio_)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(range(1, 51), pca.explained_variance_ratio_, 'o-', ms=3, label='Per-PC')
axes[0].plot(range(1, 51), explained_var, 's-', ms=3, label='Cumulative')
axes[0].axhline(0.8, color='red', linestyle='--', alpha=0.5, label='80% threshold')
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance Ratio')
axes[0].set_title('PCA — Scree Plot (Training Set)')
axes[0].legend(fontsize=8)
```

```
palette = sns.color_palette('tab20', n_classes)
color_map = {cls: palette[i] for i, cls in enumerate(class_names)}
for cls in class_names:
    mask = train_labels == cls
    axes[1].scatter(X_train_pca[mask, 0], X_train_pca[mask, 1],
                    c=[color_map[cls]], s=2, alpha=0.5, label=cls)

axes[1].set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
axes[1].set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
axes[1].set_title('PCA — PC1 vs PC2 (Training Set)')
legend = axes[1].legend(markerscale=4, fontsize=7, bbox_to_anchor=(1.01, 1), loc='upper left')

plt.tight_layout()
plt.savefig('fig_pca.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved fig_pca.png')
```

```
Running PCA on training set...
```



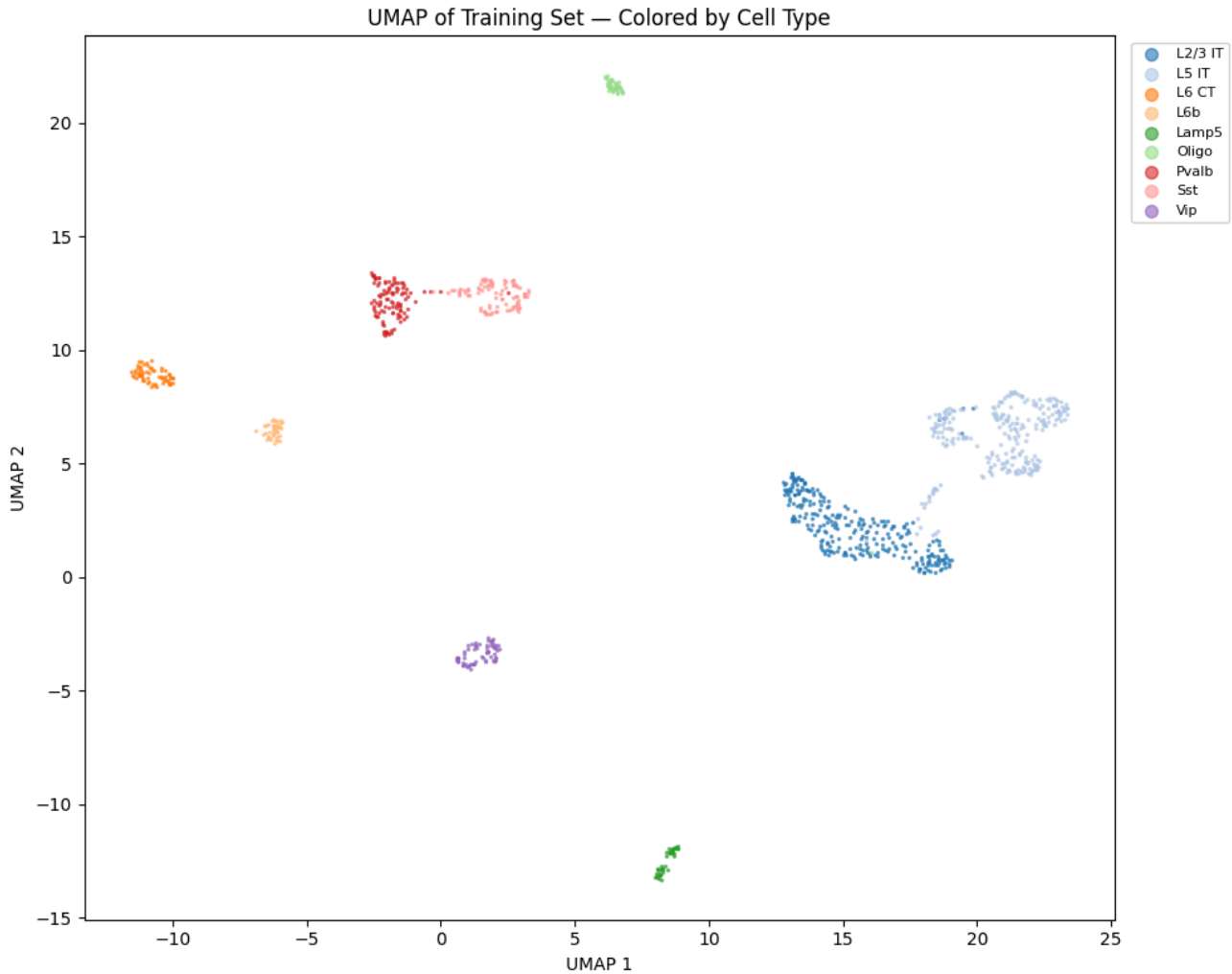

```
Saved fig_pca.png
```

```
MAX_UMAP_CELLS = 1000
if len(X_train_pca) > MAX_UMAP_CELLS:
    umap_idx = np.random.choice(len(X_train_pca), MAX_UMAP_CELLS, replace=False)
    X_umap_input = X_train_pca[umap_idx]
    umap_labels  = train_labels[umap_idx]
    print(f'  Subsampled to {MAX_UMAP_CELLS:,} cells for UMAP speed.')
else:
    X_umap_input = X_train_pca
    umap_labels  = train_labels

reducer = umap.UMAP(n_components=2, n_neighbors=30, min_dist=0.3,
                    random_state=SEED, verbose=True)
X_umap = reducer.fit_transform(X_umap_input)

fig, ax = plt.subplots(figsize=(10, 8))
for cls in class_names:
    mask = umap_labels == cls
    if mask.sum() == 0:
        continue
    ax.scatter(X_umap[mask, 0], X_umap[mask, 1],
               c=[color_map[cls]], s=2, alpha=0.6, label=cls)

ax.set_xlabel('UMAP 1')
ax.set_ylabel('UMAP 2')
ax.set_title('UMAP of Training Set — Colored by Cell Type')
ax.legend(markerscale=5, fontsize=8, bbox_to_anchor=(1.01, 1), loc='upper left')
plt.tight_layout()
plt.savefig('fig_umap.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved fig_umap.png')
```

```
  Subsampled to 1,000 cells for UMAP speed.
UMAP(min_dist=0.3, n_jobs=1, n_neighbors=30, random_state=42, verbose=True)
Fri Mar 13 07:02:46 2026 Construct fuzzy simplicial set
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting
  warn(
Fri Mar 13 07:02:47 2026 Finding Nearest Neighbors
Fri Mar 13 07:02:47 2026 Finished Nearest Neighbor Search
Fri Mar 13 07:02:47 2026 Construct embedding
```

Epochs completed: 100%| 500/500 [00:02]

```
    completed  0  /  500 epochs
    completed  50  /  500 epochs
    completed  100  /  500 epochs
    completed  150  /  500 epochs
    completed  200  /  500 epochs
    completed  250  /  500 epochs
    completed  300  /  500 epochs
    completed  350  /  500 epochs
    completed  400  /  500 epochs
    completed  450  /  500 epochs
Fri Mar 13 07:02:50 2026 Finished embedding
```

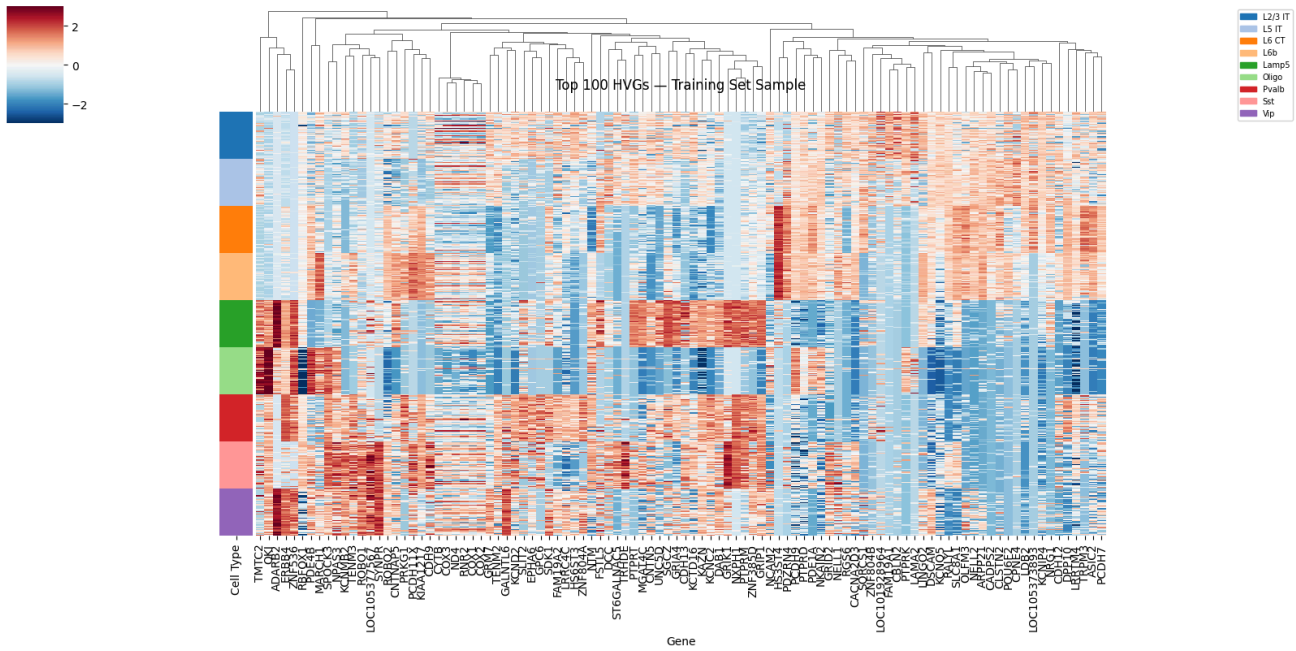



```
Saved fig_umap.png
```

```
N_HEATMAP_GENES  = 100
N_CELLS_PER_TYPE = 50

rng = np.random.default_rng(SEED)
sampled_idx, sampled_types = [], []
for i, cls in enumerate(class_names):
    cls_idx = np.where(train_labels == cls)[0]
    chosen = rng.choice(cls_idx, size=min(N_CELLS_PER_TYPE, len(cls_idx)), replace=False)
    sampled_idx.extend(chosen)
    sampled_types.extend([cls] * len(chosen))

sampled_idx   = np.array(sampled_idx)
```

```
sampled_types = np.array(sampled_types)
sort_order    = np.argsort(sampled_types)
sampled_idx   = sampled_idx[sort_order]
sampled_types = sampled_types[sort_order]

heat_data = X_train_s[sampled_idx, :N_HEATMAP_GENES]  # top-variance HVGs

type_colors = [color_map[t] for t in sampled_types]
row_colors  = pd.Series(sampled_types, name='Cell Type').map(color_map)

g = sns.clustermap(
    pd.DataFrame(heat_data, columns=gene_names_hvg[:N_HEATMAP_GENES]),
    row_colors=row_colors,
    col_cluster=True, row_cluster=False,
    cmap='RdBu_r', center=0, vmin=-3, vmax=3,
    xticklabels=True, yticklabels=False,
    figsize=(14, 8)
)
g.ax_heatmap.set_title(f'Top {N_HEATMAP_GENES} HVGs — Training Set Sample', pad=20)
g.ax_heatmap.set_xlabel('Gene')
# Add legend patches
handles = [mpatches.Patch(color=color_map[cls], label=cls) for cls in class_names]
g.ax_col_dendrogram.legend(handles=handles, fontsize=7,
                           bbox_to_anchor=(1.15, 1), loc='upper left')
plt.savefig('fig_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved fig_heatmap.png')
```

```
/usr/local/lib/python3.12/dist-packages/seaborn/matrix.py:560: UserWarning: Clustering large matrix with scipy. Insta
  warnings.warn(msg)
```

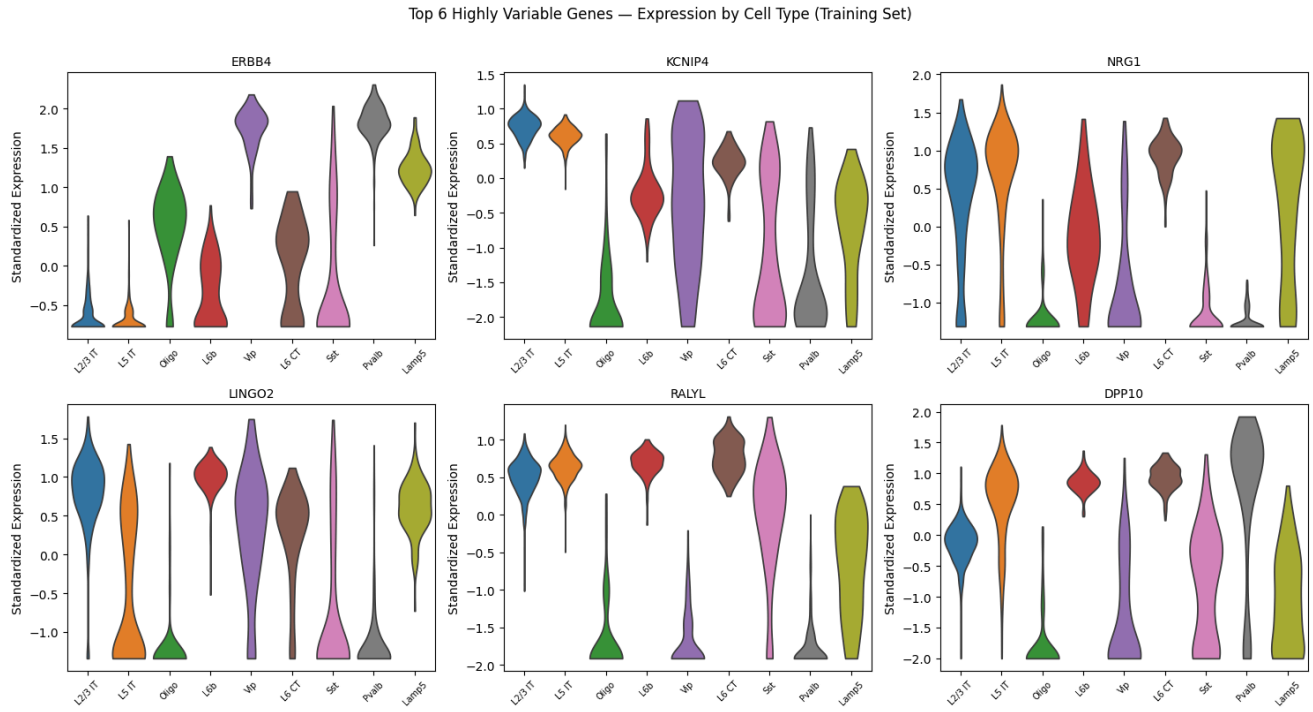



```
Saved fig_heatmap.png
```

```
print('Plotting violin plots for top 6 most variable genes...')
TOP_N_GENES = 6
fig, axes = plt.subplots(2, 3, figsize=(15, 8))

for ax, gene_idx in zip(axes.flat, range(TOP_N_GENES)):
    gene_name = gene_names_hvg[gene_idx]
    df_gene = pd.DataFrame({'expression': X_train_s[:, gene_idx],
                            'cell_type':  train_labels})
    sns.violinplot(data=df_gene, x='cell_type', y='expression',
                   hue='cell_type', inner=None, ax=ax,density_norm='width', cut=0)
    ax.set_title(f'{gene_name}', fontsize=10)
    ax.set_xlabel('')
    ax.set_ylabel('Standardized Expression')
    ax.tick_params(axis='x', rotation=45, labelsize=7)

plt.suptitle('Top 6 Highly Variable Genes — Expression by Cell Type (Training Set)',
```

```
             y=1.01, fontsize=12)
plt.tight_layout()
plt.savefig('fig_violin.png', dpi=150, bbox_inches='tight')
plt.show()
print('Saved fig_violin.png')
```

```
Plotting violin plots for top 6 most variable genes...
```




```
Saved fig_violin.png
```

7. PyTorch Dataset and DataLoaders

```
class GeneExpressionDataset(Dataset):
    """Single-cell RNA-seq dataset for 1D CNN.

    Each sample is a 1D array of shape (N_HVG,) which is
    treated as a 1-channel sequence of length N_HVG.
    """
    def __init__(self, X: np.ndarray, y: np.ndarray):
        self.X = torch.from_numpy(X[:, np.newaxis, :])  # add channel dim
        self.y = torch.from_numpy(y).long()

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

```
BATCH_SIZE = 128

train_ds = GeneExpressionDataset(X_train_s, y_train)
val_ds   = GeneExpressionDataset(X_val_s,   y_val)
test_ds  = GeneExpressionDataset(X_test_s,  y_test)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,  num_workers=0, pin_memory=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False, num_workers=0, pin_memory=True)
test_loader  = DataLoader(test_ds,  batch_size=BATCH_SIZE, shuffle=False, num_workers=0, pin_memory=True)

print(f'Train batches: {len(train_loader)} | Val batches: {len(val_loader)} | Test batches: {len(test_loader)}')
x_sample, y_sample = next(iter(train_loader))
print(f'Batch X shape: {x_sample.shape}  |  y shape: {y_sample.shape}')
```

```
Train batches: 18 | Val batches: 3 | Test batches: 3
Batch X shape: torch.Size([128, 1, 200])  |  y shape: torch.Size([128])
```

8. 1D CNN Architecture

```
Input: (batch, 1, 2000)
  Conv1d(1→64, k=15) → BN → GELU → MaxPool(4)   → (batch, 64, 497)
  Conv1d(64→128, k=9) → BN → GELU → MaxPool(4)   → (batch, 128, 123)
  Conv1d(128→256, k=7) → BN → GELU → MaxPool(4)  → (batch, 256, 30)
  Conv1d(256→256, k=5) → BN → GELU → AdaptiveAvgPool → (batch, 256, 1)
  Flatten → Linear(256→256) → GELU → Dropout(0.4)
  Linear(256→n_classes)
```

```
class ConvBlock(nn.Module):
    """Conv1d → BatchNorm → GELU → optional MaxPool."""
    def __init__(self, in_ch, out_ch, kernel, pool=None, padding='same'):
        super().__init__()
        layers = [
            nn.Conv1d(in_ch, out_ch, kernel_size=kernel, padding=padding, bias=False),
            nn.BatchNorm1d(out_ch),
            nn.Dropout(0.1),
            nn.GELU(),
        ]
        if pool:
            layers.append(nn.MaxPool1d(pool))
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return self.block(x)


class CellTypeCNN(nn.Module):
    def __init__(self, seq_len: int, n_classes: int, dropout: float = 0.4):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv1d(1, 64, kernel_size=15, padding='same', bias=False),
            ConvBlock(64,   128,  kernel=15, pool=4),
            ConvBlock(128,  256, kernel=9,  pool=4),
            ConvBlock(256, 512, kernel=7,  pool=4),
            ConvBlock(512, 512, kernel=5),
            nn.AdaptiveAvgPool1d(1),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512, 200),
            nn.ReLU(),
            nn.BatchNorm1d(200),
            nn.Linear(200, n_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

9. Training

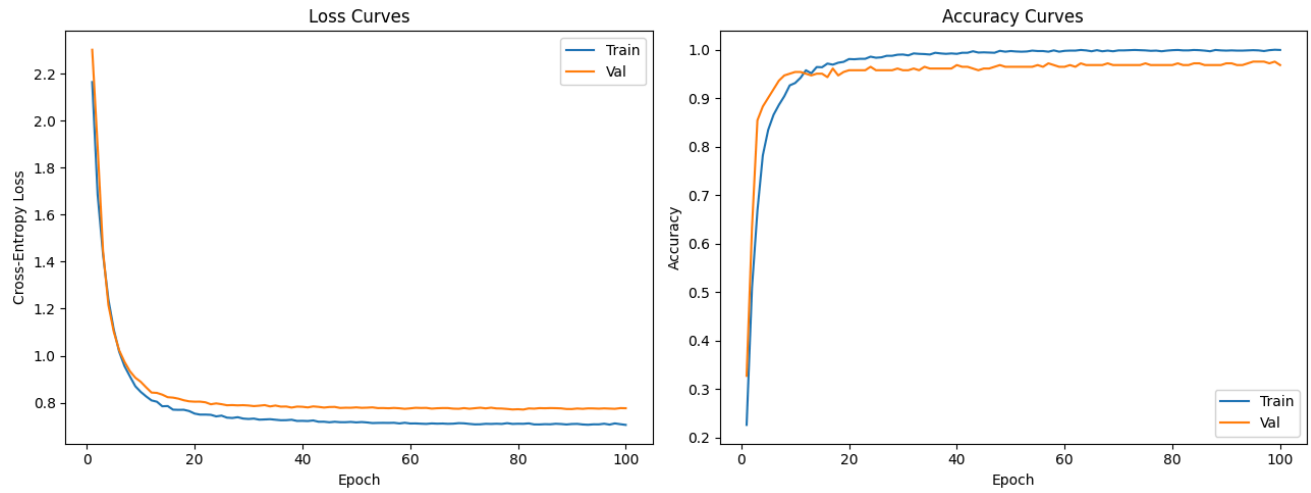
```
N_HVG = 200
```

```
model = CellTypeCNN(seq_len=N_HVG, n_classes=n_classes).to(DEVICE)
total_params = sum(p.numel() for p in model.parameters())
trainable    = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Model parameters: {total_params:,} total  |  {trainable:,} trainable')
print(model)
class_counts = np.bincount(y_train, minlength=n_classes).astype(np.float32)
class_weights = torch.tensor(1.0 / (class_counts + 1e-6), dtype=torch.float32).to(DEVICE)
class_weights = class_weights / class_weights.sum() * n_classes  # normalize

criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.1)
```

```
EPOCHS    = 100
LR        = 6e-6
WEIGHT_DECAY = 1e-6
optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS, eta_min=1e-6)

history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
best_val_acc  = 0.0

run_name = f'hvg{N_HVG}_bs{BATCH_SIZE}_epochs{EPOCHS}_lr{LR}_wd{WEIGHT_DECAY}{pd.Timestamp.now().strftime('%Y%m%d_%H
best_ckpt = f'best_model_{run_name}.pt'
writer = SummaryWriter(log_dir=f'runs/{run_name}')

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    ctx = torch.enable_grad() if train else torch.no_grad()
    with ctx:
        for xb, yb in loader:
            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
            if train:
                optimizer.zero_grad()
            logits = model(xb)
            loss   = criterion(logits, yb)
            if train:
                loss.backward()
                nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                optimizer.step()
            total_loss += loss.item() * len(yb)
            correct    += (logits.argmax(1) == yb).sum().item()
            total      += len(yb)
    return total_loss / total, correct / total


print(f'Training for {EPOCHS} epochs on {DEVICE}...')
print(f'{"Epoch":>6} | {"Train Loss":>10} | {"Train Acc":>9} | {"Val Loss":>9} | {"Val Acc":>8} | LR')
print('-' * 72)

patience = 15
min_delta = 0.0001
best_val_loss = float('inf')
epochs_no_improve = 2

for epoch in range(1, EPOCHS + 1):
    tr_loss, tr_acc = run_epoch(train_loader, train=True)
    vl_loss, vl_acc = run_epoch(val_loader,   train=False)
    scheduler.step()

    history['train_loss'].append(tr_loss)
    history['val_loss'].append(vl_loss)
    history['train_acc'].append(tr_acc)
    history['val_acc'].append(vl_acc)

    if vl_loss < best_val_loss - min_delta:
        best_val_loss = vl_loss
        best_val_acc = vl_acc
        torch.save(model.state_dict(), best_ckpt)
        epochs_no_improve = 0
        flag = '<'
    else:
        flag = ''

    lr_now = scheduler.get_last_lr()[0]
    print(f'{epoch:>6} | {tr_loss:>10.4f} | {tr_acc:>8.4f} | {vl_loss:>9.4f} | {vl_acc:>8.4f} | {lr_now:.2e}{flag}')
    writer.add_scalar('Loss/train epocj', tr_loss, global_step=epoch)
    writer.add_scalar('Loss/val epoch', vl_loss, global_step=epoch)
    writer.add_scalar('Accuracy/train epoch', tr_acc, global_step=epoch)
    writer.add_scalar('Accuracy/val epoch', vl_acc, global_step=epoch)

    if epochs_no_improve >= patience:
        print(f'\nEarly stopping triggered after {patience} epochs with no improvement in validation loss.')
        break

print(f'\nBest validation accuracy: {best_val_acc:.4f}')
```

```
     8 |     0.9119 |    0.9038 |     0.9338 |    0.9466 | 5.92e-06<
     9 |     0.8699 |    0.9261 |     0.9061 |    0.9502 | 5.90e-06<
    10 |     0.8460 |    0.9310 |     0.8889 |    0.9537 | 5.88e-06<
    11 |     0.8267 |    0.9416 |     0.8652 |    0.9537 | 5.85e-06<
    12 |     0.8097 |    0.9572 |     0.8431 |    0.9502 | 5.82e-06<
    13 |     0.8040 |    0.9510 |     0.8416 |    0.9466 | 5.79e-06<
    14 |     0.7847 |    0.9639 |     0.8342 |    0.9502 | 5.76e-06<
    15 |     0.7856 |    0.9635 |     0.8234 |    0.9502 | 5.73e-06<
    16 |     0.7702 |    0.9710 |     0.8215 |    0.9431 | 5.69e-06<
    17 |     0.7695 |    0.9688 |     0.8170 |    0.9609 | 5.65e-06<
    18 |     0.7698 |    0.9728 |     0.8102 |    0.9466 | 5.61e-06<
    19 |     0.7642 |    0.9746 |     0.8057 |    0.9537 | 5.57e-06<
    20 |     0.7535 |    0.9804 |     0.8042 |    0.9573 | 5.52e-06<
    21 |     0.7492 |    0.9800 |     0.8044 |    0.9573 | 5.48e-06
    22 |     0.7493 |    0.9808 |     0.8017 |    0.9573 | 5.43e-06<
    23 |     0.7484 |    0.9808 |     0.7933 |    0.9573 | 5.38e-06<
    24 |     0.7416 |    0.9853 |     0.7969 |    0.9644 | 5.32e-06
    25 |     0.7448 |    0.9831 |     0.7935 |    0.9573 | 5.27e-06
    26 |     0.7363 |    0.9840 |     0.7890 |    0.9573 | 5.21e-06<
    27 |     0.7348 |    0.9871 |     0.7897 |    0.9573 | 5.15e-06
    28 |     0.7382 |    0.9871 |     0.7885 |    0.9573 | 5.09e-06<
    29 |     0.7323 |    0.9893 |     0.7894 |    0.9609 | 5.03e-06
    30 |     0.7307 |    0.9898 |     0.7879 |    0.9573 | 4.97e-06<
    31 |     0.7322 |    0.9880 |     0.7853 |    0.9573 | 4.91e-06<
    32 |     0.7277 |    0.9920 |     0.7872 |    0.9609 | 4.84e-06
    33 |     0.7286 |    0.9911 |     0.7896 |    0.9573 | 4.77e-06
    34 |     0.7297 |    0.9906 |     0.7841 |    0.9644 | 4.70e-06<
    35 |     0.7274 |    0.9898 |     0.7877 |    0.9609 | 4.63e-06
    36 |     0.7252 |    0.9933 |     0.7832 |    0.9609 | 4.56e-06<
    37 |     0.7255 |    0.9920 |     0.7837 |    0.9609 | 4.49e-06
    38 |     0.7273 |    0.9911 |     0.7792 |    0.9609 | 4.42e-06<
    39 |     0.7223 |    0.9920 |     0.7833 |    0.9609 | 4.35e-06
    40 |     0.7226 |    0.9911 |     0.7825 |    0.9680 | 4.27e-06
    41 |     0.7219 |    0.9933 |     0.7798 |    0.9644 | 4.20e-06
    42 |     0.7242 |    0.9933 |     0.7841 |    0.9644 | 4.12e-06
    43 |     0.7189 |    0.9964 |     0.7821 |    0.9609 | 4.05e-06
    44 |     0.7190 |    0.9938 |     0.7792 |    0.9573 | 3.97e-06
    45 |     0.7166 |    0.9942 |     0.7815 |    0.9609 | 3.89e-06
    46 |     0.7188 |    0.9938 |     0.7819 |    0.9609 | 3.81e-06
    47 |     0.7169 |    0.9933 |     0.7776 |    0.9644 | 3.74e-06<
    48 |     0.7169 |    0.9973 |     0.7785 |    0.9680 | 3.66e-06
    49 |     0.7181 |    0.9955 |     0.7784 |    0.9644 | 3.58e-06
    50 |     0.7161 |    0.9969 |     0.7800 |    0.9644 | 3.50e-06
    51 |     0.7176 |    0.9960 |     0.7781 |    0.9644 | 3.42e-06
    52 |     0.7158 |    0.9955 |     0.7787 |    0.9644 | 3.34e-06
    53 |     0.7132 |    0.9960 |     0.7800 |    0.9644 | 3.26e-06
    54 |     0.7139 |    0.9978 |     0.7769 |    0.9644 | 3.19e-06<
    55 |     0.7141 |    0.9969 |     0.7771 |    0.9680 | 3.11e-06
    56 |     0.7138 |    0.9969 |     0.7762 |    0.9644 | 3.03e-06<
    57 |     0.7144 |    0.9955 |     0.7775 |    0.9715 | 2.95e-06
    58 |     0.7116 |    0.9982 |     0.7762 |    0.9680 | 2.88e-06
    59 |     0.7145 |    0.9955 |     0.7738 |    0.9644 | 2.80e-06<
    60 |     0.7114 |    0.9973 |     0.7758 |    0.9644 | 2.73e-06
    61 |     0.7117 |    0.9978 |     0.7782 |    0.9680 | 2.65e-06
    62 |     0.7106 |    0.9978 |     0.7774 |    0.9644 | 2.58e-06
    63 |     0.7097 |    0.9991 |     0.7778 |    0.9715 | 2.51e-06
    64 |     0.7117 |    0.9982 |     0.7746 |    0.9680 | 2.44e-06
    65 |     0.7107 |    0.9964 |     0.7762 |    0.9680 | 2.37e-06
    66 |     0.7112 |    0.9987 |     0.7771 |    0.9680 | 2.30e-06
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

epochs_range = range(1, EPOCHS + 1)

axes[0].plot(epochs_range, history['train_loss'], label='Train')
axes[0].plot(epochs_range, history['val_loss'],   label='Val')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Cross-Entropy Loss')
axes[0].set_title('Loss Curves')
axes[0].legend()

axes[1].plot(epochs_range, history['train_acc'], label='Train')
axes[1].plot(epochs_range, history['val_acc'],   label='Val')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].set_title('Accuracy Curves')
axes[1].legend()

plt.tight_layout()
plt.savefig('fig_training_curves.png', dpi=150)
plt.show()
print('Saved fig_training_curves.png')
```




```
Saved fig_training_curves.png
```

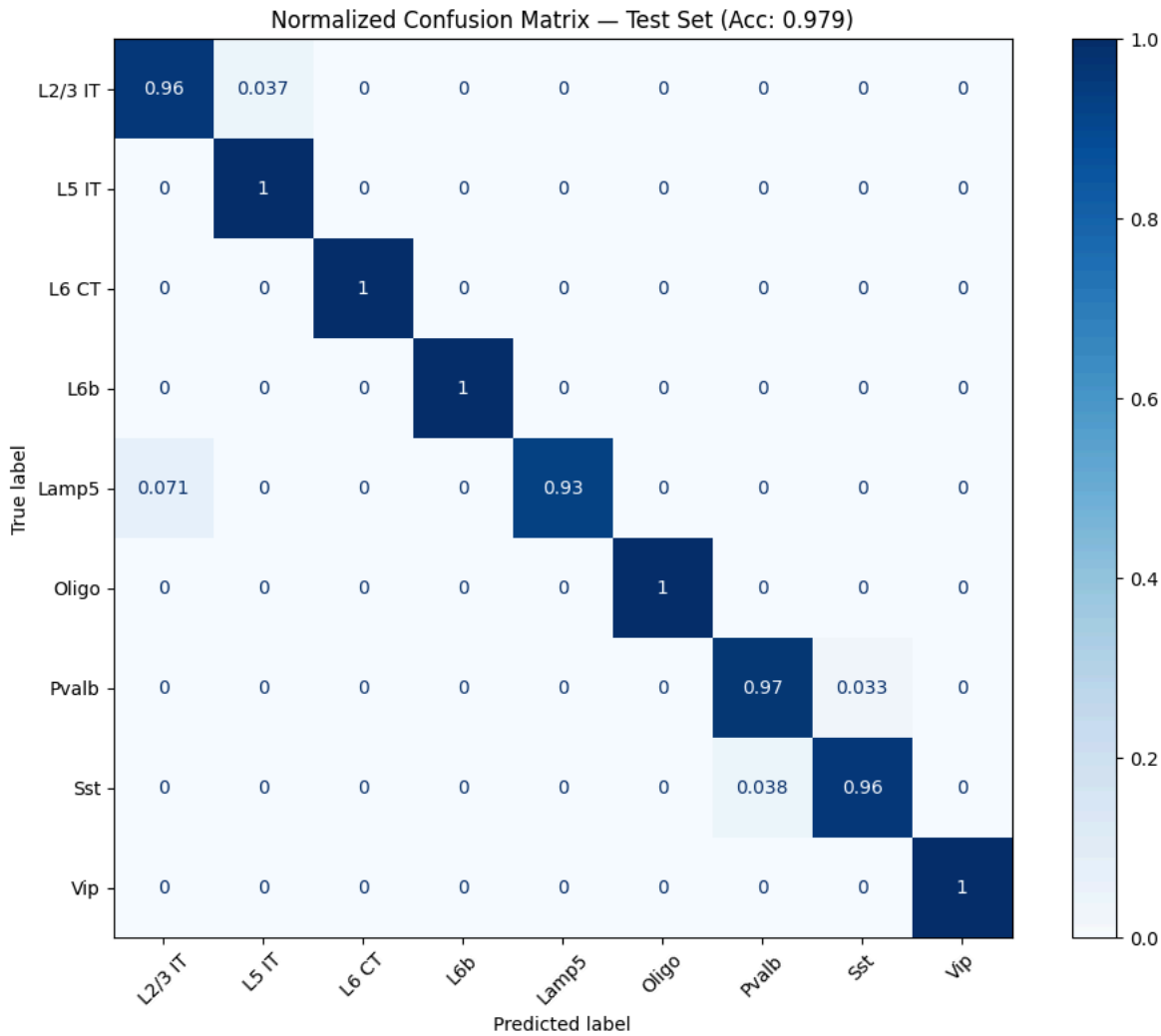
10. Test Set Evaluation

```
model.load_state_dict(torch.load(best_ckpt, map_location=DEVICE))
model.eval()

all_preds, all_true = [], []
with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(DEVICE)
        logits = model(xb)
        preds  = logits.argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_true.extend(yb.numpy())

all_preds = np.array(all_preds)
all_true  = np.array(all_true)
test_acc  = (all_preds == all_true).mean()

print(f'Test Accuracy: {test_acc:.4f} ({test_acc*100:.2f}%)')
print()
print(classification_report(all_true, all_preds,
                            target_names=class_names, digits=3))
```

```
Test Accuracy: 0.9786 (97.86%)

              precision    recall  f1-score   support

     L2/3 IT      0.988     0.963     0.975        82
       L5 IT      0.959     1.000     0.979        70
       L6 CT      1.000     1.000     1.000        19
         L6b      1.000     1.000     1.000        13
       Lamp5      1.000     0.929     0.963        14
       Oligo      1.000     1.000     1.000        10
       Pvalb      0.967     0.967     0.967        30
         Sst      0.962     0.962     0.962        26
         Vip      1.000     1.000     1.000        17

    accuracy                          0.979       281
   macro avg      0.986     0.980     0.983       281
weighted avg      0.979     0.979     0.979       281
```

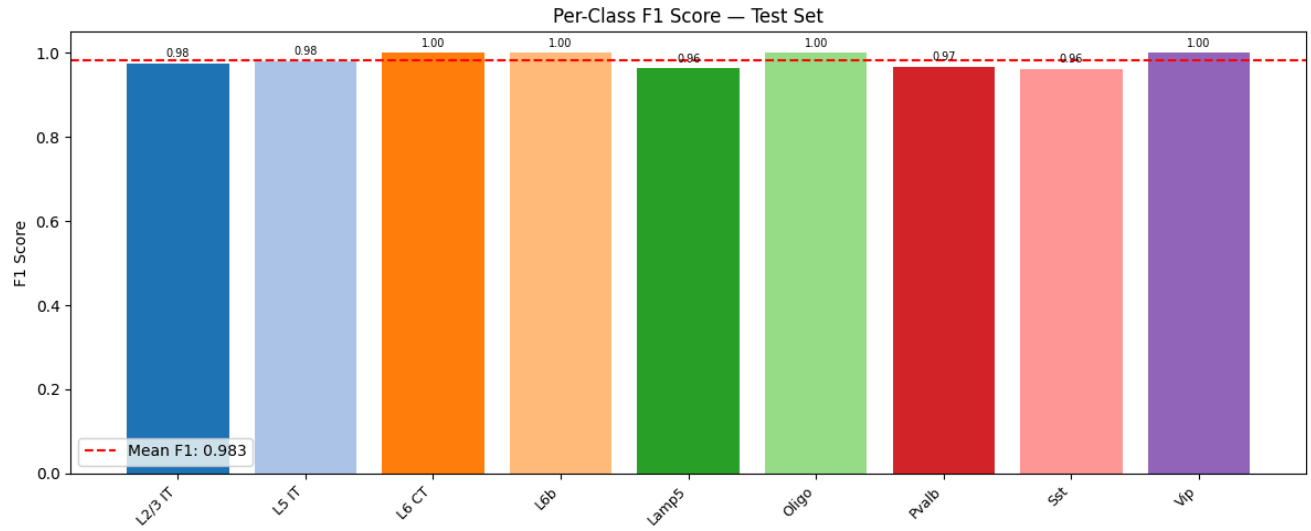
```
cm = confusion_matrix(all_true, all_preds)
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)

fig, ax = plt.subplots(figsize=(max(10, n_classes), max(8, n_classes - 2)))
disp = ConfusionMatrixDisplay(confusion_matrix=cm_norm, display_labels=class_names)
disp.plot(ax=ax, cmap='Blues', colorbar=True, xticks_rotation=45)
ax.set_title(f'Normalized Confusion Matrix — Test Set (Acc: {test_acc:.3f})')
plt.tight_layout()
plt.savefig('fig_confusion_matrix.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
print('Saved fig_confusion_matrix.png')
```




```
Saved fig_confusion_matrix.png
```

```
from sklearn.metrics import f1_score
f1_per_class = f1_score(all_true, all_preds, average=None)

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.bar(range(n_classes), f1_per_class,
              color=[color_map[c] for c in class_names])
ax.set_xticks(range(n_classes))
ax.set_xticklabels(class_names, rotation=45, ha='right', fontsize=9)
ax.set_ylabel('F1 Score')
ax.set_title('Per-Class F1 Score — Test Set')
ax.axhline(np.mean(f1_per_class), color='red', linestyle='--',
           label=f'Mean F1: {np.mean(f1_per_class):.3f}')
ax.legend()
ax.set_ylim(0, 1.05)
for bar, f1 in zip(bars, f1_per_class):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
            f'{f1:.2f}', ha='center', va='bottom', fontsize=7)
plt.tight_layout()
plt.savefig('fig_f1_per_class.png', dpi=150)
plt.show()
print('Saved fig_f1_per_class.png')
```




```
Saved fig_f1_per_class.png
```

Summary

```
from sklearn.metrics import f1_score, balanced_accuracy_score

macro_f1  = f1_score(all_true, all_preds, average='macro')
weighted_f1 = f1_score(all_true, all_preds, average='weighted')
bal_acc   = balanced_accuracy_score(all_true, all_preds)

summary = pd.DataFrame({
    'Metric': ['Test Accuracy', 'Balanced Accuracy', 'Macro F1', 'Weighted F1',
               'Best Val Accuracy', 'Total Parameters'],
    'Value':  [f'{test_acc:.4f}', f'{bal_acc:.4f}', f'{macro_f1:.4f}',
               f'{weighted_f1:.4f}', f'{best_val_acc:.4f}', f'{total_params:,}']
})
print(summary.to_string(index=False))
summary
```

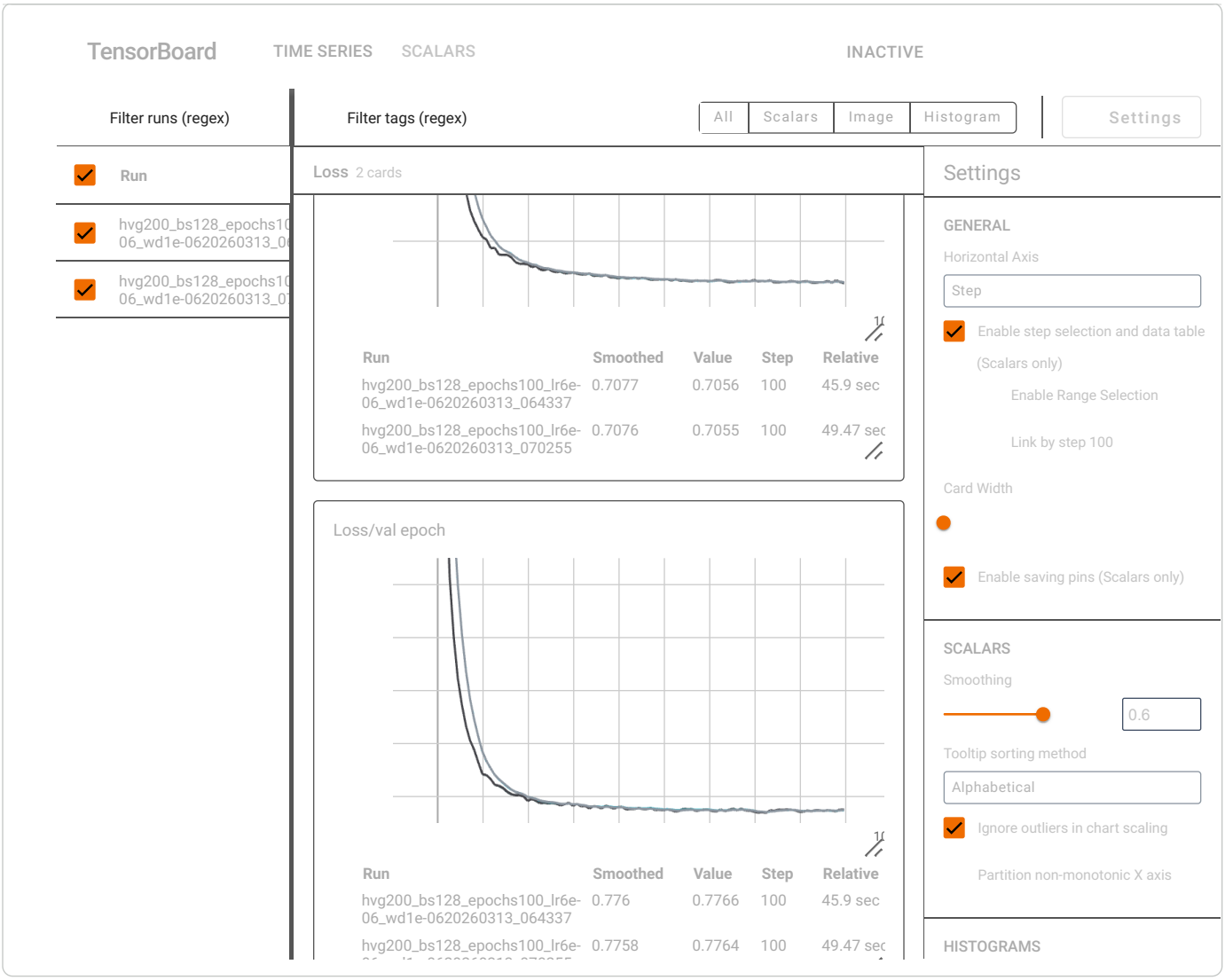
```
           Metric     Value
    Test Accuracy    0.9786
Balanced Accuracy    0.9800
         Macro F1    0.9828
      Weighted F1    0.9786
Best Val Accuracy    0.9715
 Total Parameters 2,754,601
```

	Metric	Value
0	Test Accuracy	0.9786
1	Balanced Accuracy	0.9800
2	Macro F1	0.9828
3	Weighted F1	0.9786
4	Best Val Accuracy	0.9715
5	Total Parameters	2,754,601

Next steps: Generate code with summary New interactive sheet

```
%load_ext tensorboard
```

```
%tensorboard --logdir ./runs
```




```
import pickle

with open('smartseq_inference_params.pkl', 'wb') as f:
    pickle.dump({
        'gene_names_hvg': gene_names_hvg,
        'scaler':         scaler,
        'le':             le,
        'class_names':    class_names,
        'n_classes':      n_classes,
        'n_hvg':          N_HVG,
        'best_ckpt':      best_ckpt,
        'hyperparameters': {
            'epochs':        EPOCHS,
            'learning_rate': LR,
            'weight_decay':  WEIGHT_DECAY,
            'batch_size':    BATCH_SIZE,
            'n_hvg':         N_HVG,
            'run_name':      run_name,
        },
    }, f)

print('Saved smartseq_inference_params.pkl')
```

```
Saved smartseq_inference_params.pkl
```

Secondary DataSet

```
import gc
import numpy as np
import pandas as pd
```

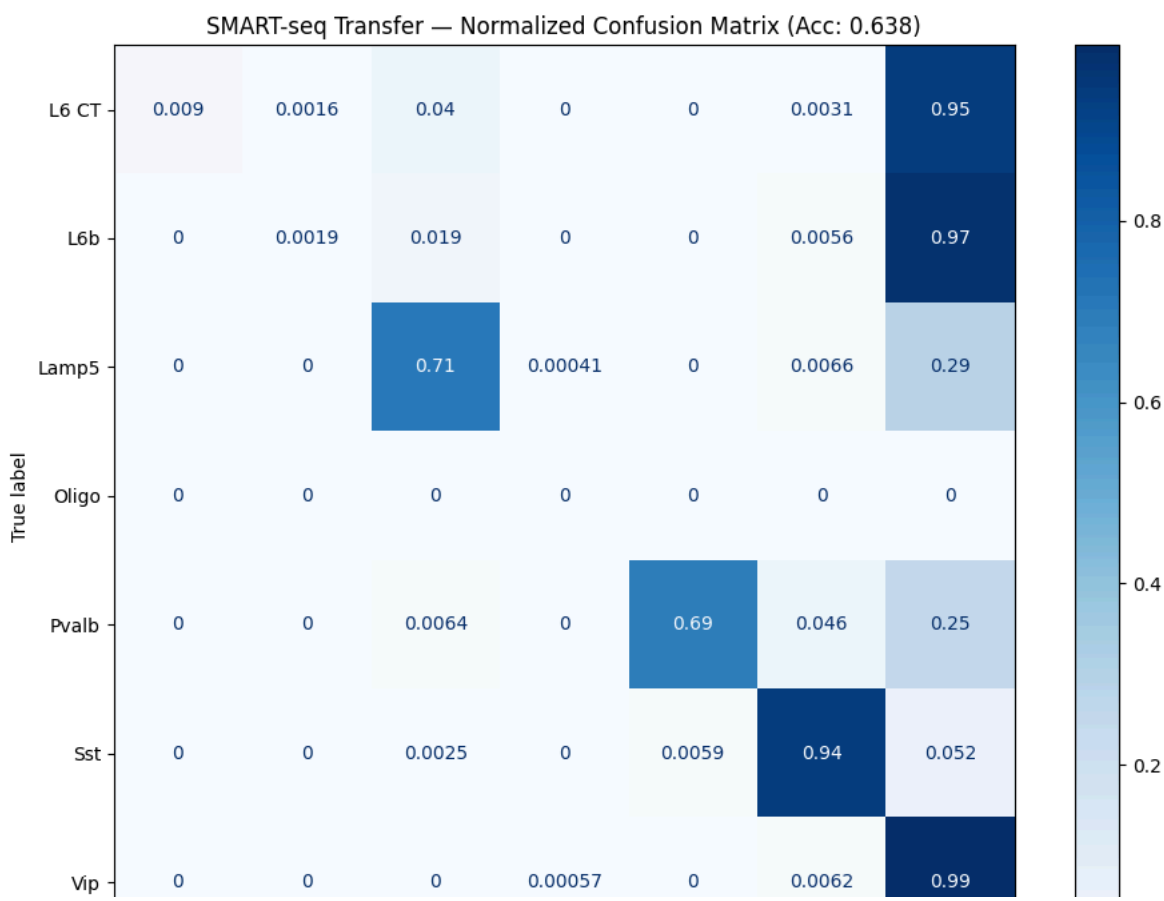
```
import pandas as pd
import polars as pl
import matplotlib.pyplot as plt
import torch
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

import torch.nn as nn
NEW_MATRIX_PATH = '/content/drive/MyDrive/FinalProject/HumanMultipleCorticalAreasSMARTseq_matrix.csv'
NEW_META_PATH   = '/content/drive/MyDrive/FinalProject/HumanMultipleCorticalAreasSMARTseq_meta.csv'
BEST_CKPT_PATH  = '/content/best_model_hvg200_bs128_epochs100_lr6e-06_wd1e-0620260313_053225.pt'

# Cells sent to GPU per forward pass — reduce if you hit GPU OOM
INFER_BATCH = 256


# ---------------------------------------------------------------------------
# 1. Free training data
# ---------------------------------------------------------------------------
class ConvBlock(nn.Module):
    """Conv1d → BatchNorm → GELU → optional MaxPool."""
    def __init__(self, in_ch, out_ch, kernel, pool=None, padding='same'):
        super().__init__()
        layers = [
            nn.Conv1d(in_ch, out_ch, kernel_size=kernel, padding=padding, bias=False),
            nn.BatchNorm1d(out_ch),
            nn.Dropout(0.1),
            nn.GELU(),
        ]
        if pool:
            layers.append(nn.MaxPool1d(pool))
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return self.block(x)


class CellTypeCNN(nn.Module):
    def __init__(self, seq_len: int, n_classes: int, dropout: float = 0.4):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv1d(1, 64, kernel_size=15, padding='same', bias=False),
            ConvBlock(64,   128,  kernel=15, pool=4),
            ConvBlock(128,  256, kernel=9,  pool=4),
            ConvBlock(256, 512, kernel=7,  pool=4),
            ConvBlock(512, 512, kernel=5),
            nn.AdaptiveAvgPool1d(1),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512, 200),
            nn.ReLU(),
            nn.BatchNorm1d(200),
            nn.Linear(200, n_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))

def free_training_data(caller_globals: dict) -> None:
    """
    Delete known training-data variables from the caller's namespace.
    Pass globals() from the notebook cell so this works correctly.
    """
    vars_to_del = [
        'expr_raw', 'expr', 'expr_filtered',
        'X_train_raw', 'X_val_raw', 'X_test_raw', 'X_temp_raw',
        'X_train_norm', 'X_val_norm', 'X_test_norm',
        'X_train_hvg', 'X_val_hvg', 'X_test_hvg',
        'meta', 'meta_indexed',
    ]
    freed = [v for v in vars_to_del if v in caller_globals]
    for v in freed:
        del caller_globals[v]
    gc.collect()
    print(f'Freed {len(freed)} training variable(s) from memory.')


def load_matrix(matrix_path: str, gene_names_hvg: list) -> tuple:
```

```
    """
    Use pl.read_csv(columns=...) to load only sample_name + HVG gene columns.
    Polars' column projection skips unwanted columns at the parser level.

    Returns
    -------
    df          : pl.DataFrame  (only the requested columns)
    found_genes : list[str]     (HVG genes actually present in this file)
    has_sid     : bool          (whether 'sample_name' column exists)
    """
    # Read 0 rows to discover the schema cheaply
    all_cols = pl.read_csv(matrix_path, n_rows=0).columns
    print(f'Total columns in new matrix: {len(all_cols):,}')

    col_set     = set(all_cols)
    found_genes = [g for g in gene_names_hvg if g in col_set]
    missing     = [g for g in gene_names_hvg if g not in col_set]
    has_sid     = 'sample_name' in col_set
    cols_to_load = (['sample_name'] if has_sid else []) + found_genes

    print(f'HVG genes found: {len(found_genes)}/{len(gene_names_hvg)}')
    if missing:
        suffix = '...' if len(missing) > 20 else ''
        print(f'Missing genes ({len(missing)}): {missing[:20]}{suffix}')

    print(f'Loading {len(cols_to_load)} columns with pl.read_csv(columns=...)...')
    df = pl.read_csv(
        matrix_path,
        columns=cols_to_load,
        null_values=['', 'NA', 'NaN'],
    )
    print(f'Loaded: {df.shape[0]:,} cells x {df.shape[1]:,} columns')
    return df, found_genes, has_sid


def load_and_align_metadata(
    meta_path: str,
    df: pl.DataFrame,
    found_genes: list,
    gene_names_hvg: list,
    has_sid: bool,
) -> tuple:
    """
    Load metadata with pl.read_csv(columns=...), inner-join with the
    expression frame on sample_name, and return:
      - arr          : np.ndarray float32, shape (n_cells, len(gene_names_hvg))
      - labels_series: pd.Series of subclass_label strings

    Genes absent from this file stay as zeros in arr.
    """
    meta = pl.read_csv(
        meta_path,
        columns=['sample_name', 'subclass_label'],
        null_values=['', 'NA', 'NaN'],
    ).filter(pl.col('subclass_label').is_not_null())

    # Inner join keeps only cells present in both files
    if has_sid:
        df = df.join(meta, on='sample_name', how='inner')
    else:
        # Fallback: positional alignment (rare)
        n = min(len(df), len(meta))
        df = df[:n].with_columns(meta['subclass_label'][:n])

    del meta
    gc.collect()

    labels_series = df['subclass_label'].to_pandas()
    drop_cols     = [c for c in ('sample_name', 'subclass_label') if c in df.columns]
    df            = df.drop(drop_cols)
    print(f'Aligned: {df.shape[0]:,} cells, {labels_series.nunique()} cell types')

    # Build ordered float32 numpy array (zeros for genes missing from file)
    gene_index = {g: i for i, g in enumerate(gene_names_hvg)}
    arr = np.zeros((df.shape[0], len(gene_names_hvg)), dtype=np.float32)
    for gene in found_genes:
        arr[:, gene_index[gene]] = (
            df[gene].cast(pl.Float32).fill_null(0.0).to_numpy()
```

```
            df[gene].cast(pl.Float32).fill_null(0.0).to_numpy()
        )

    del df
    gc.collect()
    return arr, labels_series


def preprocess(arr: np.ndarray, scaler) -> np.ndarray:
    """
    Apply library-size normalisation + log1p + z-score scaling in-place.
    `arr` is the float32 matrix returned by load_and_align_metadata.
    """
    lib = arr.sum(axis=1, keepdims=True)
    lib[lib == 0] = 1.0
    arr /= lib
    arr *= 1e4
    np.log1p(arr, out=arr)    # in-place, no extra allocation

    arr[:] = scaler.transform(arr)  # copy result back into arr
    print(f'Preprocessed shape: {arr.shape}')
    return arr


def run_inference(X_scaled, model_class, ckpt_path, n_hvg, n_classes, device, batch_size=INFER_BATCH):
    """
    Load the best checkpoint, run batched inference, and return raw predicted
    integer labels.
    """
    model = model_class(seq_len=n_hvg, n_classes=n_classes).to(device)
    model.load_state_dict(torch.load(ckpt_path, map_location=device))
    model.eval()

    preds = []
    with torch.no_grad():
        for i in range(0, len(X_scaled), batch_size):
            batch = torch.from_numpy(X_scaled[i:i + batch_size, np.newaxis, :]).to(device)
            preds.append(model(batch).argmax(1).cpu().numpy())

    del X_scaled
    gc.collect()
    return np.concatenate(preds)


def evaluate(new_preds, new_pred_labels, labels_series, le, class_names):
    """
    Case-insensitive evaluation: map SMART-seq labels (e.g. VIP, SST)
    to training labels (Vip, Sst) before scoring.
    """
    # Build case-insensitive mapping: lowered training name → original name
    train_lower = {c.lower(): c for c in class_names}

    # Map each SMART-seq label to its training-set equivalent (or keep as-is)
    label_map = {lbl: train_lower.get(lbl.lower(), lbl) for lbl in labels_series.unique()}
    mapped_labels = labels_series.map(label_map)

    known_mask = mapped_labels.isin(class_names)
    n_matched  = known_mask.sum()
    print(f'\nCells with known class labels: {n_matched:,} / {len(labels_series):,}')

    # Show which SMART-seq labels were mapped
    mapped_pairs = {k: v for k, v in label_map.items() if k != v and v in class_names}
    if mapped_pairs:
        print(f'Case-mapped labels: {mapped_pairs}')

    unknown = sorted(set(mapped_labels.unique()) - set(class_names))
    if unknown:
        print(f'Unknown classes ({len(unknown)}): {unknown}')

    if n_matched == 0:
        print('No overlapping classes found.')
        return

    true_known = mapped_labels[known_mask].values
    pred_known = new_pred_labels[known_mask.values]
    true_enc   = le.transform(true_known)
    pred_enc   = le.transform(pred_known)
```

```
    acc = (true_enc == pred_enc).mean()
    print(f'Accuracy on known classes: {acc:.4f} ({acc * 100:.2f}%)\n')

    present_enc   = np.unique(np.concatenate([true_enc, pred_enc]))
    present_names = le.inverse_transform(present_enc)

    print(classification_report(
        true_enc, pred_enc,
        labels=present_enc,
        target_names=present_names,
        digits=3,
        zero_division=0,
    ))

    cm = confusion_matrix(true_enc, pred_enc, labels=present_enc)
    row_sum = cm.sum(axis=1, keepdims=True)
    cm_norm = np.zeros_like(cm, dtype=float)
    np.divide(cm, row_sum, out=cm_norm, where=row_sum != 0)

    dim = max(10, len(present_enc))
    fig, ax = plt.subplots(figsize=(dim, max(8, dim - 2)))
    ConfusionMatrixDisplay(confusion_matrix=cm_norm, display_labels=present_names).plot(
        ax=ax, cmap='Blues', colorbar=True, xticks_rotation=45)
    ax.set_title(f'SMART-seq Transfer — Normalized Confusion Matrix (Acc: {acc:.3f})')
    plt.tight_layout()
    plt.savefig('fig_smartseq_confusion_matrix.png', dpi=150, bbox_inches='tight')
    plt.show()


def end_to_end(
    gene_names_hvg,
    scaler,
    le,
    class_names,
    model_class,
    n_classes: int,
    device,
    caller_globals: dict = None,
    matrix_path: str = NEW_MATRIX_PATH,
    meta_path: str   = NEW_META_PATH,
    ckpt_path: str   = BEST_CKPT_PATH,
    infer_batch: int = INFER_BATCH,
) -> None:
    if caller_globals is not None:
        free_training_data(caller_globals)
    df, found_genes, has_sid = load_matrix(matrix_path, gene_names_hvg)
    arr, labels_series = load_and_align_metadata(
        meta_path, df, found_genes, gene_names_hvg, has_sid)
    X = preprocess(arr, scaler)

    # 3. Inference
    raw_preds   = run_inference(X, model_class, ckpt_path,
                                n_hvg=len(gene_names_hvg),
                                n_classes=n_classes,
                                device=device,
                                batch_size=infer_batch)
    del X; gc.collect()

    pred_labels = le.inverse_transform(raw_preds)
    print(f'\nPredictions for {len(raw_preds):,} cells:')
    print(pd.Series(pred_labels).value_counts())

    evaluate(raw_preds, pred_labels, labels_series, le, class_names)
```

```
import pickle
with open('smartseq_inference_params.pkl', 'rb') as f:
    p = pickle.load(f)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
end_to_end(
    gene_names_hvg = p['gene_names_hvg'],
    scaler         = p['scaler'],
    le             = p['le'],
    class_names    = p['class_names'],
    model_class    = CellTypeCNN,
    n_classes      = p['n_classes'],
    device         = device,
```

```
    device          device,
    ckpt_path      = p['best_ckpt'])
```

```
Total columns in new matrix: 50,282
HVG genes found: 200/200
Loading 201 columns with pl.read_csv(columns=...)...
Loaded: 49,417 cells x 201 columns
Aligned: 47,432 cells, 19 cell types
Preprocessed shape: (47432, 200)

Predictions for 47,432 cells:
Vip        36534
Sst         3592
Lamp5       2895
Oligo       2257
Pvalb       1961
L2/3 IT      164
L6 CT         23
L6b            6
Name: count, dtype: int64

Cells with known class labels: 14,761 / 47,432
Case-mapped labels: {'VIP': 'Vip', 'LAMP5': 'Lamp5', 'SST': 'Sst', 'PVALB': 'Pvalb'}
Unknown classes (13): ['Astrocyte', 'Endothelial', 'IT', 'L4 IT', 'L5 ET', 'L5/6 IT Car3', 'L5/6 NP', 'Microglia',
Accuracy on known classes: 0.6377 (63.77%)

              precision    recall  f1-score   support

       L6 CT      1.000     0.009     0.018      2556
         L6b      0.333     0.002     0.004      1080
       Lamp5      0.921     0.706     0.799      2434
       Oligo      0.000     0.000     0.000         0
       Pvalb      0.993     0.695     0.817      2800
         Sst      0.924     0.939     0.932      2358
         Vip      0.413     0.993     0.583      3533

    accuracy                          0.638     14761
   macro avg      0.655     0.478     0.450     14761
weighted avg      0.784     0.638     0.579     14761
```